CSC209: Software Tools and Systems Programming

A3 Project Report

Monte Carlo Simulator

Submitted by

Group 51548

Anvi Singh, Ching-Yu (Jenny) Hwang, Christina Hu

I. Project Overview

Our program is **Category 1: Multi-Process Application Using Pipes** that performs a Monte Carlo simulation, which is a statistical method used to estimate the expected values of a user-defined function involving randomness by repeatedly sampling from specified probability distributions. The user interacts with the program through the console, entering a formula that may include the variables **x, y, z, w, or v** using only standard arithmetic operations (+, -, *, /, ^). Examples: x*y + z^2 or w^3-v. Trigonometric or other advanced functions are **not supported**.

The formula entered must satisfy the following constraints to expect a valid output:

1. Avoid dividing by variables that could be zero
2. Choose distributions that won't produce extreme values for your formula
3. Ensure the formula has a finite expected value so Monte Carlo converges

For each variable, the user selects a distribution (**Uniform, Normal, Exponential, Bernoulli, Binomial, or Poisson**) and provides any necessary parameters (e.g., bounds for Uniform). The user also specifies the total number of simulations to run. The program outputs the total trials ran, estimated expected value E[f], expected value squared E[f²], and variance Var[f].

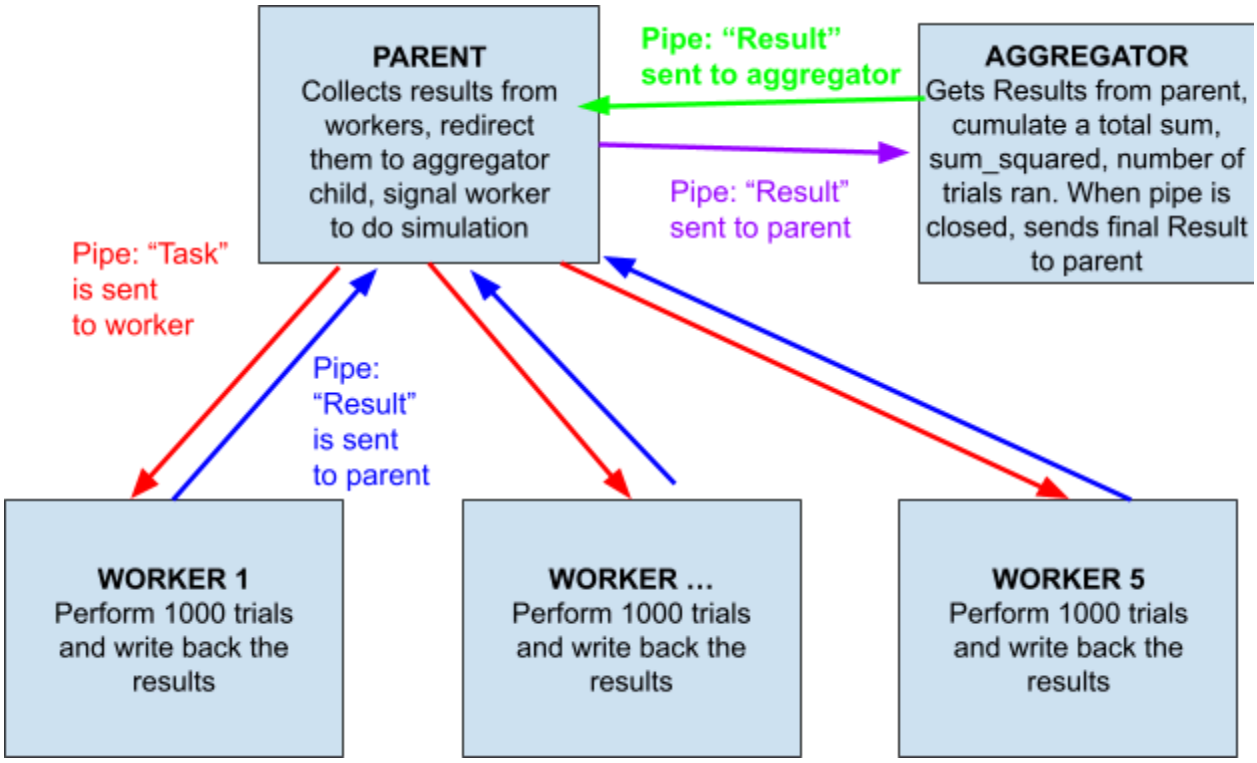
The program distributes work across a fixed pool of worker processes via pipes. Each worker receives a task containing the formula, variable distributions, and a chunk of samples to run. Workers draw values from the specified distributions, evaluate the formula, and return partial results to the parent, which forwards them to a dedicated aggregator process that accumulates the totals. The parent redistributes remaining work to idle workers until all simulations are complete. Crashed workers are automatically detected, respawned, and their last task reissued. "Running" signals are sent periodically during computation to reassure the user that the simulation is progressing normally.

II. Build Instruction

1. After making sure you are at the right directory, type **make** in the console
2. Type **./monte_carlo**
3. You can now start interacting with the program by inputting the function you wish to estimate and other required information that the program asks you for.

III. Project Design

Project Architecture Diagram



Communication Protocols

Message 1: Worker telling parent result of simulation

Field	Description
Direction	Worker -> Parent
Encoding	Result is a struct containing the result of the simulation(s) the child ran; the number of trials that were run, the sum of their results, and the sum of their results squared. typedef struct { long trials; double sum; double sum_sq; } Result;
Semantics	Every worker will run its own simulation(s) according to the message it received, and upon completion, write its result back to

	the parent.
Error Handling	If write returns -1, the parent logs the error, marks the child as dead, and creates a replacement child which will attempt a simulation again (or a less complex option would be to just print error and terminate worker).

Message 2: Parent signaling worker to do simulation (task)

Field	**Description**
Direction	Parent -> Worker

Encoding	**Task** is a struct containing information about the chunk of simulations to perform, the variables and corresponding distributions, the number of variables, and the formula user wants to estimate. VarDist struct contains information about the variable distribution and one about the chunk of simulations to perform.
	typedef struct Task {
	long num_samples;
	char formula[256]; // e.g. "x*y^2"
	VarDist vars[MAXVAR]; // each variable with its distribution
	// + params
	int num_vars;
	} Task;

Semantics	The parent sends each worker a task describing how many samples to generate and the distributions to sample from. Worker executes the simulations and returns a Result message when done.
Error Handling	If write fails, the parent logs the error and may either resend the task to the same worker or mark the worker as dead and assign the task to another worker.

Additionally, each variable's distribution type and parameters are stored in a **VarDist** struct embedded within the **Task** struct. It is never sent independently between processes, so it wasn't included as part of the communication protocol.

```
typedef struct VariableDistribution {
   char     variable;   // the character name, e.g. 'x'
   int      distribution; // the distribution, indicated by number
   double   param1; // the parameter 1, if it exists
   double   param2; // the parameter 2, if it exists, otherwise it's -1
} VarDist;
```

IV. Concurrency Model

(*main.c: 95-106*) After parsing user input (formula, variables, distributions, and number of trials N), the parent sets workers_num = min(5, N) (*main.c: 92-93*) to avoid spawning more workers than tasks. Before spawning workers, the parent sets up two pipes (one for sending Tasks to the worker and one for receiving Results) and defines arrays such as pid_t pids, int busy, Task last_task, two arrays of worker pipes (to_worker[][2], from_worker[][2]), all of size workers_num for worker management, and then it calls spawn_workers workers_num times to create the worker pool.

(*main.c: 677-712*) Inside the spawn_workers function, it calls fork(), enters the child process, wires its pipes to the parent to stdin and stdout using dup2() so the worker process can read Tasks from stdin and write Results to stdout to the parent. Then execl("./worker") is called to switch from child to worker process. This function also modifies the pids array so each element refers to the pid of each child.

(*main.c: 118-132*) We then enter the first task dispatch loop, where for each worker, we create a shallow copy of the Task struct (template) into variable t, and as long as we have samples remaining, we write Task t (with t.num_samples = (if the remaining samples is less than CHUNK, = remaining_sum, otherwise CHUNK)), to the worker. Then we change the index of the worker in busy array to 1 to indicate it is busy, and track the task sent to it in last_task[i], and increase total_tasks_dispatched by 1

(*main.c: 135-275*) Then we enter the select loop, where we keep all workers busy (first loop condition) and finish all remaining samples (second loop condition) (loop condition: total_tasks_done < total_tasks_dispatched || remaining_num > 0).

> (*main.c: 139-161*) The first part of this loop is crash recovery, where the program loops through all busy workers (busy[i] == 1) and calls waitpid with WNOHANG argument to check whether it has terminated. Because the current code hasn't reached where the parent calls to terminate the children, terminating here can only mean crashing. waitpid returning >0 means the child terminated unexpectedly, in which case the zombie child is collected by the waitpid call, the pipes are closed, and a child is created from which a new worker process is spawned by spawn_process helper to replace the dead worker. The last task given to the dead worker is then sent to it too. If there was any error in writing the task to the new process, the program terminates.

(*main.c: 164-255*) A loop adds every busy worker's read pipe to fd_set so select() can watch all of them simultaneously. int maxfd tracks the highest file descriptor in the set (so select() knows where to stop scanning). The parent knows a worker is ready to be read from when select() returns and FD_ISSET confirms that worker's pipe has data. Therefore, after select() returns (after line 184), we enter a loop to read data from all ready workers. It goes through all workers and uses FD_ISSET to which out which ones have data to be read. For those that are busy and have data, it first reads the tag byte to check if it's a result or a heartbeat message, then reads the Result from the worker. Otherwise (not busy or have no data), the loop continues. Then, if the tag was a heartbeat, the parent prints a message and moves on. If the tag was a result, the parent marks it as no longer busy, forwards the result to the aggregator, and if there is remaining work it immediately redispatches a new task to that same worker and marks it busy again. Then the parent removes that worker's fd from the set so the second loop doesn't read the same worker again in the same iteration.

We use select here because if we loop through workers one by one, the parent would be blocked if one worker is slow. Select() blocks on all pipes and only returns when at least one is ready, so the parent never gets stuck waiting on a single worker.

(*main.c: 259-271*) After processing all busy workers, we enter a loop that only processes idle workers and sends them tasks.

(*main.c: 278-296*) Here is after we exit the select loop, where all tasks have been dispatched and results collected. We enter a loop to send a signal to the worker process to shut down, after which we close the pipes and collect the child with waitpid.

(*main.c: 301-333*) After collecting all children, we close the to_aggregator[1] pipe, read the final result it sent through the pipe, compute the average estimate (E[X]), E[X^2], and Var[X], close the from_aggregator[0] pipe, and waitpid to collect the aggregator process zombie. Finally, we print the results to the user.

We chose a fixed worker pool over spawning a new process per chunk because process creation is expensive and spawning many processes in parallel can overwhelm the system. A worker pool keeps resource usage predictable while still achieving parallelism.

V. Error Handling and Robustness

1. Worker crashes mid-computation. If a worker process dies unexpectedly, the parent detects it in two ways. Before each select(), the parent calls waitpid(WNOHANG) on every busy worker. If a worker has died, it returns the worker's PID. Additionally, if the worker's pipe closes unexpectedly, read() returns 0 (EOF). In both cases, the parent closes the crashed worker's pipes, spawns a replacement via fork() and execl("./worker"), and resends the original Task so no samples are lost.

2. Invalid user input. If the user enters an invalid distribution number, out-of-range parameters (e.g. negative lambda, standard deviation ≤ 0, probability outside [0,1]), or a formula containing disallowed variables, the parent catches these during parsing before any workers are spawned. get_distributions() validates all parameters with strtol/strtod and loops until valid input is received. parse_variables() exits with an error message if an invalid variable is detected. No malformed data is ever passed to workers.

3. fgets returning NULL. If fgets returns NULL during input collection due to EOF (e.g. Ctrl+D) or an unexpected stdin closure, the parent detects this immediately at every fgets call and exits with an error message before any processes are spawned or any resources are allocated.

4. Pipe creation or dup2 failure. In spawn_process(), every pipe() and dup2() call is checked. If any pipe or dup2 fails, the program exits immediately with a descriptive error rather than continuing with a broken file descriptor.

5. Write failures on pipes. Every write() to a worker or aggregator is checked against the expected byte count. If a write returns fewer bytes than expected, such as when the reader has closed the pipe, the parent catches it and exits with an error rather than silently sending incomplete data.

6. fork() or execl() failure. In spawn_process(), both fork() and execl() are checked. If fork() returns -1 or execl() returns at all (it only returns on failure), the program exits immediately with a descriptive error message.

7. Aggregator returning zero trials. After the aggregator sends back its final Result, the parent checks that trials is nonzero before computing the final estimate. This guards against a division by zero when computing E[f].

VI. Implementation Details

Formula Parsing and Validation

parse_variables() iterates through the formula and looks for allowed variables"xyzwv" (main.c, lines 345-382). get_distributions() prompts the user for a distribution for each variable and validates all parameters before accepting input (main.c, lines 384-673).

Formula Evaluation

evaluate.c uses a recursive parser, it handles operator precedence across four levels. addition/subtraction, multiplication/division, exponentiation, and parentheses (evaluate.c, lines 30-110).

Worker

Each worker reads Tasks from stdin in a loop, samples each variable from its assigned distribution, evaluates the formula, and accumulates the results into a local Result struct (worker.c, lines 45-95). Upon completing its chunk it writes a MSG_RESULT tag followed by the Result back to the parent (worker.c, lines 97-105). A shutdown signal of num_samples=0 causes the worker to exit cleanly (worker.c, line 52).

Worker Pool and Process Spawning

The parent spawns k = min(5, N) workers via fork() and execl("./worker") inside spawn_process(), which sets up two pipes per worker wired to stdin and stdout using dup2() (main.c, lines 510-545). Workers persist for the lifetime of the program, receiving multiple Task chunks via their internal read loop (worker.c, lines 45-105).

Dynamic Work Queue and Running Signals

The parent immediately redispatches a new Task to any worker that finishes its chunk (main.c, lines 210-225). Workers send a MSG_HEARTBEAT tag every 500 samples, prompting the parent to print "worker is running..." (worker.c, lines 96-99, main.c, lines 193-195).

Crash Detection and Recovery

The parent checks every busy worker using waitpid(WNOHANG) before each select() call (main.c, lines 145-165). If a crash is detected, pipes are closed, a replacement is spawned, and the original Task stored in last_task[i] is resent (main.c, lines 155-168).

Aggregation and Shutdown

Each completed Result is sent to the aggregator, which accumulates totals until the parent closes the pipe (aggregator.c, lines 16-36). The parent then reads the final combined Result, computes E[f], E[f²], and Var[f], and prints them (main.c, lines 290-329). Workers and the aggregator are reaped via waitpid() to prevent zombies (main.c, lines 255-295).

VII. Contributions

Jenny: Brainstormed the project idea and architecture with Anvi, collaborated on communication protocols, and hosted team meetings. I wrote the user prompting and input parsing logic, implemented evaluate.c for recursive formula evaluation, wrote the Makefile, and co-wrote protocols.c with Anvi defining the Task, VarDist, and Result structs. Also proposed and implemented two additional features: worker crash detection and recovery and heartbeat monitoring so the parent can signal user still workers are still running normally during computation. I wrote tests, debugged, and cleaned up code with Anvi.

Anvi: I brainstormed the idea for our project along with Jenny. The three of us collaborated to complete the optional Project Proposal, for which I especially helped with deciding the communication protocols, defining the program inputs and general steps, and drawing the architecture diagram. During the programming of the project, I was mainly responsible for creating the functionality for the workers through the worker.c file, which the parent uses for fork() + exec(). I worked with Jenny to resolve major merge conflicts and debugged much of the program after merging everything together. I also wrote a majority of the Project Report for final submission.

Christina: Collaborated with the team to complete the Project Proposal and Project Report (section 4). I wrote the overall workflow, worker pool set up and wiring. I also wrote the select loop, which keeps idle workers busy until all N simulations are completed, and implemented the aggregator child process file, to which the parent pipes the worker pool results. After all workers are finished, it computes the final estimated expected value and variance and sends it back to the parent. I wrote the distribution and parameter selection part of the user prompting and input parsing. I also added error checks and cleaned the code, and wro